For example, if the text T is `unreferenced`, with $n = 12$, then the text T' is `unreferenced@decnerefernu`, with $n' = 25$ and the following suffix array and LCP array:

i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
$T'[i]$	u	n	r	e	f	e	r	e	n	c	e	d	@	d	e	c	n	e	r	e	f	e	r	n	u
$SA[i]$	13	10	16	12	14	15	11	4	20	8	18	6	22	5	21	9	17	2	24	3	19	7	23	25	1
$LCP[i]$	0	0	1	0	1	0	1	1	4	1	1	3	2	0	3	0	1	1	1	0	5	2	1	0	1

The maximum LCP value is achieved at $LCP[21] = 5$, and $SA[20] = 3 = n' - SA[21] - LCP[21] + 2$. The suffixes of T' starting at indices $SA[20]$ and $SA[21]$ are `referenced@decnerefernu` and `refernu`, both of which start with the length-5 palindrome `refer`.

Alas, this method is not foolproof. Give an input string T that causes this method to give results that are shorter than the longest palindrome contained within T, and explain why your input causes the method to fail.

Problems

32-1 String matching based on repetition factors

Let y^i denote the concatenation of string y with itself i times. For example, $(\texttt{ab})^3 = \texttt{ababab}$. We say that a string $x \in \Sigma^*$ has ***repetition factor*** r if $x = y^r$ for some string $y \in \Sigma^*$ and some $r > 0$. Let $\rho(x)$ denote the largest r such that x has repetition factor r.

a. Give an efficient algorithm that takes as input a pattern $P[1{:}m]$ and computes the value $\rho(P[{:}i])$ for $i = 1, 2, \ldots, m$. What is the running time of your algorithm?

b. For any pattern $P[1{:}m]$, let $\rho^*(P)$ be defined as $\max \{\rho(P[{:}i]) : 1 \le i \le m\}$. Prove that if the pattern P is chosen randomly from the set of all binary strings of length m, then the expected value of $\rho^*(P)$ is $O(1)$.

c. Argue that the procedure REPETITION-MATCHER correctly finds all occurrences of pattern $P[1{:}m]$ in text $T[1{:}n]$ in $O(\rho^*(P)n + m)$ time. (This algorithm is due to Galil and Seiferas. By extending these ideas

greatly, they obtained a linear-time string-matching algorithm that uses only $O(1)$ storage beyond what is required for P and T.)

```
REPETITION-MATCHER(T, P, n, m)
 1 k = 1 + ρ*(P)
 2 q = 0
 3 s = 0
 4 while s ≤ n − m
 5     if T[s + q + 1] == P[q + 1]
 6         q = q + 1
 7         if q == m
 8             print "Pattern occurs with shift" s
 9     if q == m or T[s + q + 1] ≠ P[q + 1]
10         s = s + max {1, ⌈q/k⌉}
11         q = 0
```

32-2 A linear-time suffix-array algorithm

In this problem, you will develop and analyze a linear-time divide-and-conquer algorithm to compute the suffix array of a text $T[1{:}n]$. As in Section 32.5, assume that each character in the text is represented by an underlying encoding, which is a positive integer.

The idea behind the linear-time algorithm is to compute the suffix array for the suffixes starting at 2/3 of the positions in the text, recursing as needed, use the resulting information to sort the suffixes starting at the remaining 1/3 of the positions, and then merge the sorted information in linear time to produce the full suffix array.

For $i = 1, 2, \ldots, n$, if i mod 3 equals 1 or 2, then i is a ***sample position***, and the suffixes starting at such positions are ***sample suffixes***. Positions 3, 6, 9, … are ***nonsample positions***, and the suffixes starting at ***nonsample*** positions are ***nonsample suffixes***.

The algorithm sorts the sample suffixes, sorts the nonsample suffixes (aided by the result of sorting the sample suffixes), and merges the sorted sample and nonsample suffixes. Using the example text $T =$

`bippityboppityboo`, here is the algorithm in detail, listing substeps of each of the above steps:

1. The sample suffixes comprise about 2/3 of the suffixes. Sort them by the following substeps, which work with a heavily modified version of T and may require recursion. In part (a) of this problem on page 999, you will show that the orders of the suffixes of T and the suffixes of the modified version of T are the same.

   A. Construct two texts P_1 and P_2 made up of "metacharacters" that are actually substrings of three consecutive characters from T. We delimit each such metacharacter with parentheses. Construct

   $$P_1 = (T[1{:}3])(T[4{:}6])(T[7{:}9]) \cdots (T[n'{:}n' + 2]),$$

   where n' is the largest integer congruent to 1, modulo 3, that is less than or equal to n and T is extended beyond position n with the special character Ø, with encoding 0. With the example text T = `bippityboppityboo`, we get that

   P_1 = (`bip`) (`pit`) (`ybo`) (`ppi`) (`tyb`) (`oo`Ø).

   Similarly, construct

   $$P_2 = (T[2{:}4])(T[5{:}7])(T[8{:}10]) \cdots (T[n''{:}n'' + 2]),$$

   where n'' is the largest integer congruent to 2, modulo 3, that is less than or equal to n. For our example, we have

   P_2 = (`ipp`) (`ity`) (`bop`) (`pit`) (`ybo`) (`o`ØØ).

position in T	1	4	7	10	13	16	2	5	8	11	14	17
metacharacter in P	(bip)	(pit)	(ybo)	(ppi)	(tyb)	(oo$\emptyset$)	(ipp)	(ity)	(bop)	(pit)	(ybo)	(o$\emptyset\emptyset$)
character in P'	1	7	10	8	9	6	3	4	2	7	10	5
position in P'	1	2	3	4	5	6	7	8	9	10	11	12
$SA_{P'}$	1	9	7	8	12	6	10	2	4	5	11	3
positions in T of sorted sample suffixes of T	1	8	2	5	17	16	11	4	10	13	14	7

Figure 32.15 Computed values when sorting the sample suffixes of the linear-time suffix-array algorithm for the text T = `bippityboppityboo`.

If n is a multiple of 3, append the metacharacter ($\emptyset\emptyset\emptyset$) to the end of P_1. In this way, P_1 is guaranteed to end with a metacharacter containing $\emptyset$. (This property helps in part (a) of this problem.) The text P_2 may or may not end with a metacharacter containing $\emptyset$.

B. Concatenate P_1 and P_2 to form a new text P. Figure 32.15 shows P for our example, along with the corresponding positions of T.

C. Sort and rank the unique metacharacters of P, with ranks starting from 1. In the example, P has 10 unique metacharacters: in sorted order, they are (`bip`), (`bop`), (`ipp`), (`ity`), (o$\emptyset\emptyset$), (oo$\emptyset$), (`pit`), (`ppi`), (`tyb`), (`ybo`). The metacharacters (`pit`) and (`ybo`) each appear twice.

D. As Figure 32.15 shows, construct a new "text" P' by renaming each metacharacter in P by its rank. If P contains k unique metacharacters, then each "character" in P' is an integer from 1 to k. The suffix arrays for P and P' are identical.

E. Compute the suffix array $SA_{P'}$ of P'. If the characters of P' (i.e., the ranks of metacharacters in P) are unique, then you can compute its suffix array directly, since the ordering of the individual characters gives the suffix array. Otherwise, recurse to compute the suffix array of P', treating the ranks in P' as the input characters in the recursive call. Figure 32.15 shows the suffix array $SA_{P'}$ for our example. Since the number of metacharacters in P, and hence the

length of P', is approximately $2n/3$, this recursive subproblem is smaller than the current problem.

F. From $SA_{P'}$ and the positions in T corresponding to the sample positions, compute the list of positions of the sorted sample suffixes of the original text T. Figure 32.15 shows the list of positions in T of the sorted sample suffixes in our example.

2. The nonsample suffixes comprise about 1/3 of the suffixes. Using the sorted sample suffixes, sort the nonsample suffixes by the following substeps.

i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
$T[i]$	b	i	p	p	i	t	y	b	o	p	p	i	t	y	b	o	o	Ø	Ø
r_i	1	3	□	8	4	□	12	2	□	9	7	□	10	11	□	6	5	0	0

Figure 32.16 The ranks r_1 through r_{n+3} for the text T = `bippityboppityboo` with $n = 17$.

G. Extending the text T by the two special characters ØØ, so that T now has $n + 2$ characters, consider each suffix $T[i:]$ for $i = 1, 2, \ldots, n + 2$. Assign a rank r_i to each suffix $T[i:]$. For the two special characters ØØ, set $r_{n+1} = r_{n+2} = 0$. For the sample positions of T, base the rank on the list of sorted sample positions of T. The rank is currently undefined for the nonsample positions of T. For these positions, set r_i = □. Figure 32.16 shows the ranks for T = `bippityboppityboo` with $n = 17$.

H. Sort the nonsample suffixes by comparing tuples $(T[i], r_{i+1})$. In our example, we get $T[15:] < T[12:] < T[9:] < T[3:] < T[6:]$ because (b, 6) < (i, 10) < (o, 9) < (p, 8) < (t, 12).

3. Merge the sorted sets of suffixes. From the sorted set of suffixes, determine the suffix array of T.

This completes the description of a linear-time algorithm for computing suffix arrays. The following parts of this problem ask you to show that

certain steps of this algorithm are correct and to analyze the algorithm's running time.

a. Define a ***nonempty suffix*** at position i of the text P created in substep B as all metacharacters from position i of P up to and including the first metacharacter of P in which Ø appears or the end of P. In the example shown in Figure 32.15, the nonempty suffixes of P starting at positions 1, 4, and 11 of P are (bip) (pit) (ybo) (ppi) (tyb) (ooØ), (ppi) (tyb) (ooØ), and (ybo) (oØØ), respectively. Prove that the order of suffixes of P is the same as the order of its nonempty suffixes. Conclude that the order of suffixes of P gives the order of the sample suffixes of T. (*Hint:* If P contains duplicate metacharacters, consider separately the cases in which two suffixes both start in P_1, both start in P_2, and one starts in P_1 and the other starts in P_2. Use the property that Ø appears in the last metacharacter of P_1.)

b. Show how to perform substep C in $\Theta(n)$ time, bearing in mind that in a recursive call, the characters in T are actually ranks in P' in the caller.

c. Argue that the tuples in substep H are unique. Then show how to perform this substep in $\Theta(n)$ time.

d. Consider two suffixes $T[i:]$ and $T[j:]$, where $T[i:]$ is a sample suffix and $T[j:]$ is a nonsample suffix. Show how to determine in $\Theta(1)$ time whether $T[i:]$ is lexicographically smaller than $T[j:]$. (*Hint:* Consider separately the cases in which $i \bmod 3 = 1$ and $i \bmod 3 = 2$. Compare tuples whose elements are characters in T and ranks as shown in Figure 32.16. The number of elements per tuple may depend on whether $i \bmod 3$ equals 1 or 2.) Conclude that step 3 can be performed in $\Theta(n)$ time.

e. Justify the recurrence $T(n) \le T(2n/3 + 2) + \Theta(n)$ for the running time of the full algorithm, and show that its solution is $O(n)$. Conclude that the algorithm runs in $\Theta(n)$ time.

32-3 Burrows-Wheeler transform

The ***Burrows-Wheeler transform***, or ***BWT***, for a text T is defined as follows. First, append a new character that compares as lexicographically less than every character of T, and denote this character by `$` and the resulting string by T'. Letting n be the length of T', create n rows of characters, where each row is one of the n cyclic rotations of T'. Next, sort the rows lexicographically. The BWT is then the string of n characters in the rightmost column, read top to bottom.

For example, let T = `rutabaga`, so that T' = `rutabaga$`. The cyclic rotations are

```
rutabaga$
utabaga$r
tabaga$ru
abaga$rut
baga$ruta
aga$rutab
ga$rutaba
a$rutabag
$rutabaga
```

Sorting the rows and numbering the sorted rows gives

```
1 $rutabaga
2 a$rutabag
3 abaga$rut
4 aga$rutab
5 baga$ruta
6 ga$rutaba
7 rutabaga$
8 tabaga$ru
9 utabaga$r
```

The BWT is the rightmost column, `agtbaa$ur`. (The row numbering will be helpful in understanding how to compute the inverse BWT.)

The BWT has applications in bioinformatics, and it can also be a step in text compression. That is because it tends to place identical characters together, as in the BWT of `rutabaga`, which places two of the instances of `a` together. When identical characters are placed

together, or even nearby, additional means of compressing become available. Following the BWT, combinations of move-to-front encoding, run-length encoding, and Huffman coding (see Section 15.3) can provide significant text compression. Compression ratios with the BWT tend improve as the text length increases.

a. Given the suffix array for T', show how to compute the BWT in $\Theta(n)$ time.

In order to decompress, the BWT must be invertible. Assuming that the alphabet size is constant, the inverse BWT can be computed in $\Theta(n)$ time from the BWT. Let's look at the BWT of `rutabaga`, denoting it by $BWT[1{:}n]$. Each character in the BWT has a unique lexicographic rank from 1 to n. Denote the rank of $BWT[i]$ by $rank[i]$. If a character appears multiple times in the BWT, each instance of the character has a rank 1 greater than the previous instance of the character. Here are BWT and $rank$ for `rutabaga`:

i	1	2	3	4	5	6	7	8	9
$BWT[i]$	a	g	t	b	a	a	$	u	r
$rank[i]$	2	6	8	5	3	4	1	9	7

For example, $rank[1] = 2$ because $BWT[1]$ = `a` and the only character that precedes the first a lexicographically is `$` (which we defined to precede all other characters, so that `$` has rank 1). Next, we have $rank[2] = 6$ because $BWT[2]$ = `g` and five characters in the BWT precede `g` lexicographically: `$`, the three instances of `a`, and `b`. Jumping ahead to $rank[5] = 3$, that is because $BWT[5]$ = `a`, and because this `a` is the second instance of `a` in the BWT, its $rank$ value is 1 greater than the $rank$ value for the previous instance of `a`, in position 1.

There is enough information in BWT and $rank$ to reconstruct T' from back to front. Suppose that you know the rank r of a character c in T'. Then c is the first character in row r of the sorted cyclic rotations. The last character in row r must be the character that precedes c in T'. But you know which character is the last character in row r, because it is $BWT[r]$. To reconstruct T' from back to front, start with `$`, which you can find in BWT. Then work backward using BWT and $rank$ to reconstruct T'.

Let's see how this strategy works for `rutabaga`. The last character of T', `$`, appears in position 7 of BWT. Since $rank[7] = 1$, row 1 of the sorted cyclic rotations of T' begins with `$`. The character that precedes `$` in T' is the last character in row 1, which is $BWT[1]$: `a`. Now we know that the last two characters of T' are `a$`. Looking up $rank[1]$, it equals 2, so that row 2 of the sorted cyclic rotations of T' begins with `a`. The last character in row 2 precedes a in T', and that character is $BWT[2]$ = `g`. Now we know that the last three characters of T' are `ga$`. Continuing on, we have $rank[2] = 6$, so that row 6 of the sorted cyclic rotations begins with `g`. The character preceding `g` in T' is $BWT[6]$ = `a`, and so the last four characters of T' are `aga$`. Because $rank[6] = 4$, `a` begins row 4 of the sorted cyclic rotations of T'. The character preceding `a` in T' is the last character in row 4, $BWT[4]$ = `b`, and the last five characters of T' are `baga$`. And so on, until all n characters of T' have been identified, from back to front.

b. Given the array $BWT[1{:}n]$, write pseudcode to compute the array $rank[1{:}n]$ in $\Theta(n)$ time, assuming that the alphabet size is constant.

c. Given the arrays $BWT[1{:}n]$ and $rank[1{:}n]$, write pseudocode to compute T' in $\Theta(n)$ time.

Chapter notes

The relation of string matching to the theory of finite automata is discussed by Aho, Hopcroft, and Ullman [5]. The Knuth-Morris-Pratt algorithm [267] was invented independently by Knuth and Pratt and by Morris, but they published their work jointly. Matiyasevich [317] earlier discovered a similar algorithm, which applied only to an alphabet with two characters and was specified for a Turing machine with a two-dimensional tape. Reingold, Urban, and Gries [377] give an alternative treatment of the Knuth-Morris-Pratt algorithm. The Rabin-Karp algorithm was proposed by Karp and Rabin [250]. Galil and Seiferas [173] give an interesting deterministic linear-time string-matching algorithm that uses only $O(1)$ space beyond that required to store the pattern and text.

The suffix-array algorithm in Section 32.5 is by Manber and Myers [312], who first proposed the notion of suffix arrays. The linear-time algorithm to compute the longest common prefix array presented here is by Kasai et al. [252]. Problem 32-2 is based on the DC3 algorithm by Kärkkäinen, Sanders, and Burkhardt [245]. For a survey of suffix-array algorithms, see the article by Puglisi, Smyth, and Turpin [370]. To learn more about the Burrows-Wheeler transform from Problem 32-3, see the articles by Burrows and Wheeler [78] and Manzini [314].

---

[1] For suffix arrays, the preprocessing time of $O(n \lg n)$ comes from the algorithm presented in Section 32.5. It can be reduced to $\Theta(n)$ by using the algorithm in Problem 32-2. The factor k in the matching time denotes the number of occurrences of the pattern in the text.

[2] We write $\Theta(n - m + 1)$ instead of $\Theta(n - m)$ because s takes on $n - m + 1$ different values. The "+1" is significant in an asymptotic sense because when $m = n$, computing the lone t_s value takes $\Theta(1)$ time, not $\Theta(0)$ time.

[3] Informally, lexicographic order is "alphabetical order" in the underlying character set. A more precise definition of lexicographic order appears in Problem 12-2 on page 327.

[4] Why keep saying "length at most"? Because for a given value of l, a substring of length l starting at position i is $T[i..i + l - 1]$. If $i + l - 1 > n$, then the substring cuts off at the end of the text.